CS5730 Operating Systems: 2011 Midterm Solutions

2011 · CS5730 Operating Systems Midterm (2011/11/14)

1. Short Answer: Your answers should be as concise as possible; aim for three or four sentences.

Q1.1 (5 pt) Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a single-processor system if the synchronization primitives are to be used in user-level programs. 2011

ANSWER

Disabling interrupts works on a uniprocessor only for **trusted kernel code**. It is the wrong tool for **user-level** synchronization because:

1. **It is a privileged instruction.** Letting user programs execute it breaks dual-mode protection; the hardware traps if user mode tries.
2. **The OS loses control of the CPU.** While interrupts are off, the **timer interrupt** cannot fire, so no preemption or context switch can happen. A buggy or malicious program could disable interrupts and never re-enable them (e.g. an infinite loop in its critical section). It would then **monopolize the CPU** and hang the whole system.
3. **Other events are delayed or lost.** I/O-completion interrupts and clock ticks are postponed, so devices sit idle, the system clock drifts and response time suffers. User critical sections can be long, which makes this worse.

(It also does not scale: on a multiprocessor it only stops the local CPU.) User programs should instead use OS-provided semaphores/mutexes or atomic hardware instructions such as test_and_set or swap.

Q1.2 (5 pt) Provide an example in which multithreading does not provide better performance than a single-threaded solution. 2011

ANSWER

Multithreading helps only if there is work that can overlap. Two examples where it does not help:

- A **strictly sequential computation**, where each step depends on the previous result. Examples are computing a tax return or a recurrence such as Fibonacci with an iterative loop. There is nothing to run in parallel, and threads only add creation, context-switch and synchronization overhead.
- A **CPU-bound program on a single processor** with no I/O (e.g. a matrix multiply). The threads just take turns on the same CPU, so the total time is the same or worse because of switching overhead.

Another standard example is a shell or interactive program that only waits for one user's input. It spends its time blocked anyway.

Q1.3 (5 pt) Show that, if the `wait()` and `signal()` semaphore operations are not executed atomically, then mutual exclusion may be violated. 2011

ANSWER

Busy-waiting definitions: `wait(S){ while (S <= 0); S--; }` and `signal(S){ S++; }`, with S = 1 (unlocked).

If wait() is not atomic, the test and the decrement can be interleaved:

Step	P0	P1	S
1	tests S<=0 → false (S=1)		1
2	*preempted*	tests S<=0 → false (S=1)	1
3		S-- ; enters critical section	0
4	S-- ; enters critical section		−1

Both P0 and P1 are in the critical section, so mutual exclusion is violated. Likewise, S-- and S++ are each load, modify, store at machine level. If two processes run them concurrently, an update can be lost, e.g. two signal() calls increment S only once. A lost signal can block a process forever, and an extra count lets two processes in. So wait() and signal() must be indivisible.

2. Consider a multiprocessor system and a multithreaded program written using the many-to-many threading model. Let the number of user-level threads in the program be more than the number of processors in the system. Discuss the performance implications of the following scenarios.

Q2.1 (5 pt) The number of kernel threads allocated to the program is less than the number of processors. 2011

ANSWER

Fewer kernel threads than processors. The kernel can schedule only that many of the program's threads at once. Some processors are left **idle** as far as this program is concerned, and the available parallelism is wasted. Also, if a kernel thread blocks (e.g. on I/O), even fewer user threads can run.

Q2.2 (5 pt) The number of kernel threads allocated to the program is equal to the number of processors. 2011

ANSWER

Equal to the number of processors. All processors can run the program's threads simultaneously, giving the best case while no thread blocks. However, when a kernel thread **blocks** (I/O, page fault, system call), its processor becomes **idle**. No spare kernel thread exists to run another ready user thread on it, so performance drops during blocking.

Q2.3 (5 pt) The number of kernel threads allocated to the program is greater than the number of processors but less than the number of userlevel threads. 2011

ANSWER

More kernel threads than processors, but fewer than user threads. This gives the best utilization. When one kernel thread blocks, the kernel can schedule **another ready kernel thread** (carrying another user thread) on that processor, so CPUs stay busy. The cost is some extra kernel overhead: more kernel threads to create, schedule and context-switch.

3. Multilevel queue scheduling

Q3.1 (2 pt) List two possible scheduling among queues, which we discussed in class. 2011

ANSWER

Two ways to schedule **among** the queues of a multilevel queue:

1. **Fixed-priority (preemptive) scheduling.** Serve all processes in a higher-priority queue (e.g. foreground/interactive) before any process in a lower queue (background/batch). A process arriving in a higher queue preempts a lower one. Lower queues may **starve**.
2. **Time slicing among queues.** Each queue gets a fixed share of CPU time, which it schedules among its own processes. Example: 80% to the foreground queue (RR) and 20% to the background queue (FCFS).

Q3.2 (8 pt) Describe how Linux support the above two possible inter-queue scheduling. 2011

ANSWER

Fixed priority between classes. Linux has two priority ranges mapped onto one global scale (lower number = higher priority). **Real-time** tasks are 0–99 and use POSIX SCHED_FIFO / SCHED_RR with static priorities. **Normal (time-sharing)** tasks are 100–139, from nice −20...+19. A runnable real-time task always runs before any normal task, which is fixed-priority scheduling between the two "queues". With CFS, the scheduler likewise picks the highest-priority **scheduling class** first (real-time class before the default class).

Time slicing. In the O(1) scheduler (2.5) each task gets a time slice based on priority, and higher priority gets a longer quantum. Each CPU's runqueue has an **active** and an **expired** priority array. A task runs while it has slice left, then moves to expired. When the active array is empty, the arrays are **swapped**, so every task gets its share. In CFS (2.6.23+), instead of fixed quanta each task gets a **proportion** of CPU time within a **target latency**, weighted by nice value. The task with the smallest **virtual runtime (vruntime)** runs next. This is proportional sharing of time.

4. Consider a logical address space of 64 pages of 1024 words each, mapped onto a physical memory of 32 frames.

Q4.1 (5 pt) How many bits are there in the logical address? 2011

ANSWER

Pages: 64 = 2^6 gives a 6-bit page number. Page size: 1024 = 2^{10} words gives a 10-bit offset.
Logical address = 6 + 10 = 16 bits.

Q4.2 (5 pt) How many bits are there in the physical address? 2011

ANSWER

Frames: 32 = 2^5 gives a 5-bit frame number. The frame size equals the page size (2^{10}), so the offset is 10 bits.
Physical address = 5 + 10 = 15 bits.

5. A monitor `M` has processes `P1` and `P2` waiting in entry queue and process `P3` waiting in signal queue. Process `P0` is active in `M`. `M` also has two condition variables `x`, `y`. `x` has processes `P4` and `P5` waiting in its corresponding queue, and `y` has process `P6` waiting in its corresponding queue. Suppose `M` implements the signal and wait semantic. List the contents of each queue and the active process after the following events happen. (Note that each event happens right after the above state; that is, these events are independent to each other.)

Q5.1 (5 pt) New process `P7` accesses `M`. 2011

ANSWER

P7 just joins the end of the entry queue. Nothing else changes.

Active in M	Entry queue	Signal queue	x queue	y queue
P0	P1, P2, **P7**	P3	P4, P5	P6

Q5.2 (5 pt) `P0` executes `x.signal()`. 2011

ANSWER

Signal-and-wait (Hoare): the first waiter on x, **P4, resumes immediately**. The signaler **P0 is suspended in the signal queue**.

Active in M	Entry queue	Signal queue	x queue	y queue
P4	P1, P2	P3, **P0**	P5	P6

Q5.3 (5 pt) `P0` executes `y.wait()`. 2011

ANSWER

P0 joins y's queue and releases the monitor. The signal queue has priority over the entry queue, so **P3 becomes active**.

Active in M	Entry queue	Signal queue	x queue	y queue
P3	P1, P2	(empty)	P4, P5	P6, **P0**

Q6 (15 pt) Discuss how the following pairs of scheduling criteria conflict in certain settings. 2011

Qa) CPU utilization and response time 2011

ANSWER

CPU utilization vs. response time. CPU utilization is highest when context switches are rare, as with FCFS or a long quantum, because each switch is pure overhead. Good response time for interactive users needs **frequent preemption** (a short RR quantum) so every user gets the CPU quickly. That causes many context switches, so the CPU spends more time switching and less doing useful work, which lowers utilization.

Qb) Average turnaround time and maximum waiting time 2011

ANSWER

Average turnaround time vs. maximum waiting time. Average turnaround is minimized by running the **shortest jobs first** (SJF/SRTF). But a long job can be postponed again and again as short jobs keep arriving, so it may **starve**. Its waiting time, the **maximum** waiting time, becomes very large (unbounded). Bounding the maximum wait (e.g. with FCFS or aging) raises the average turnaround.

Qc) I/O device utilization and CPU utilization 2011

ANSWER

I/O device utilization vs. CPU utilization. Keeping devices busy means running **I/O-bound** processes as soon as their I/O completes, so they can issue the next request quickly. This means frequently preempting CPU-bound processes, and the context-switch overhead lowers CPU utilization. Maximizing CPU utilization means letting CPU-bound jobs run long without switching, which leaves the I/O devices **idle** while I/O-bound jobs wait for the CPU.

Qd) Deadline and fairness 2011

ANSWER

Deadline vs. fairness. To meet deadlines, the scheduler must always give the CPU to the most urgent process (earliest deadline or real-time priority), regardless of how much CPU it has already had. Other processes may wait a long time or **starve**, which is unfair. A **fair** scheduler gives everyone an equal share, so urgent tasks may not get enough CPU in time and **miss their deadlines**.

Qe) Throughput and context switch overhead 2011

ANSWER

Throughput vs. context-switch overhead. Throughput (jobs completed per unit time) is improved by finishing short jobs quickly, which needs preemption: SJF/SRTF or a small RR quantum. Every preemption adds **context-switch overhead**, during which no useful work is done. Minimizing switches (FCFS, large quantum) traps short jobs behind long ones (**convoy effect**), so fewer jobs complete per unit time. If the quantum is too small, overhead itself eats the throughput.

Q7 (20 pt) Consider the following Peterson's solution for the critical region problem: 2011

```
shared int turn;              /* set to i or j */
shared boolean flag[2];       /* set to T or F */

while (TRUE) {
    ...
    flag[i] = TRUE;
    turn = j;
    while (turn == j && flag[j] == TRUE) { /* no-op */ }
    ...
    /* critical region */
    ...
    flag[i] = FALSE;
    ...
    /* remainder region */
    ...
}
```

Prove Peterson's solution satisfy the three required critical region conditions.

ANSWER

Pi enters its critical region only when `flag[j] == FALSE` or `turn == i`.

1. Mutual exclusion. Suppose P0 and P1 were both in their critical regions. Then flag[0] = flag[1] = TRUE, so each must have passed the while because turn equalled its own index. That would require turn == 0 and turn == 1 at the same time, which is impossible. More precisely, turn is a single shared variable holding the value written **last**. The process that wrote turn last set it to the other process's index, so it must wait while the other is interested. Hence at most one is inside.

2. Progress. If Pj is not trying to enter (flag[j] == FALSE, i.e. it is in its remainder region), Pi passes the while at once. If both are trying, turn is either i or j, so exactly one of them passes, and the decision is made in finite time. Only processes in their entry sections take part; a process in its remainder section cannot block anyone.

3. Bounded waiting. Suppose Pi is waiting (turn == j and flag[j] == TRUE). When Pj leaves, it sets flag[j] = FALSE, so Pi can enter. If Pj immediately tries again, it sets turn = i, so Pj now waits and Pi enters. Pi therefore waits for at most **one** entry of Pj into its critical region.

Q8 (Bonus: 20 pt) Consider the code below as a variation of Peterson's solution above: 2011

```
shared int turn;              /* set to i or j */
shared boolean flag[2];       /* set to T or F */

while (TRUE) {
    ...
    flag[i] = TRUE;
    turn = i;     /* traditionally set to j */
    while (turn == j && flag[j] == TRUE) { /* no-op */ }
    ...
    /* critical region */
    ...
    flag[i] = FALSE;
    turn = j;     /* not usually done */
    ...
    /* remainder region */
    ...
}
```

Does the above solution satisfy the three required critical region conditions? If not, which property or properties is violated?

ANSWER

No. Mutual exclusion is violated. In the entry code each process now sets turn to **its own** index, so a process that has just written turn never waits (its test `turn == j` is false). Counter-example:

Step	P0	P1
1	flag[0] = TRUE; turn = 0	
2	test turn==1? false → **enters CR**	
3	(in critical region)	flag[1] = TRUE; turn = 1
4	(in critical region)	test turn==0? false → **enters CR**

Both are in the critical region. In general a newly arriving process always gets in, even while the other is inside. (Progress and bounded waiting still hold: a waiting Pi enters as soon as Pj exits, because the exit sets turn = i. But without mutual exclusion the solution is incorrect.)